

Python Programming

A Practical Reference Guide

Core Language, Data Structures, Functions, Object-Oriented
Programming, File Handling, Error Handling, and the Standard Library

Compiled Reference Edition

Table of Contents

1. Introduction to Python
2. Installing Python and Running Code
3. Variables, Data Types, and Operators
4. Strings and String Formatting
5. Collections: Lists, Tuples, Sets, and Dictionaries
6. Control Flow: Conditionals and Loops
7. Functions
8. Modules and Packages
9. Object-Oriented Programming
10. Exception Handling
11. File Input and Output
12. Useful Standard Library Modules
13. Virtual Environments and Package Management
14. Best Practices and Style (PEP 8)
15. Quick Reference Tables

1. Introduction to Python

Python is a high-level, general-purpose programming language known for its readable syntax and broad standard library. It was first released in 1991 and has since become one of the most widely used languages for web development, data analysis, automation, scientific computing, and artificial intelligence.

1.1 Key Characteristics

- **Interpreted:** Python code is executed line by line by an interpreter, rather than being compiled to machine code ahead of time.
- **Dynamically typed:** Variable types are determined at runtime, so you do not need to declare types explicitly.
- **Multi-paradigm:** Python supports procedural, object-oriented, and functional programming styles.
- **Batteries included:** The standard library ships with modules for file I/O, networking, math, data serialization, and much more.
- **Extensive ecosystem:** Third-party packages (installed via pip) cover nearly every domain, from web frameworks like Django and Flask to data science tools like NumPy and pandas.

1.2 Philosophy

Python's design philosophy emphasizes code readability and simplicity. This is captured informally in "The Zen of Python," a collection of guiding principles accessible by running `import this` in a Python interpreter. Core ideas include preferring explicit code over implicit behavior, valuing simplicity over complexity, and readability counting.

2. Installing Python and Running Code

2.1 Installation

Python can be downloaded from the official website for Windows, macOS, and Linux. Most Linux distributions and macOS versions ship with Python pre-installed, though it is often useful to install a specific recent version separately. After installation, verify the version from a terminal:

```
$ python3 --version
Python 3.12.4
```

2.2 Running Python Code

There are three common ways to run Python code:

- **Interactive shell (REPL):** Type `python3` in a terminal to open an interactive prompt for testing small snippets.
- **Script files:** Save code in a file ending in `.py` and run it with `python3 filename.py`.
- **Notebooks:** Tools like Jupyter allow mixing code, output, and text in a single interactive document, popular for data analysis.

```
# hello.py
def greet(name):
    return f"Hello, {name}!"

if __name__ == "__main__":
    print(greet("World"))

# Run with: python3 hello.py
# Output:   Hello, World!
```

Note: The `if __name__ == "__main__":` block ensures the code inside only runs when the file is executed directly, not when it is imported as a module elsewhere.

3. Variables, Data Types, and Operators

3.1 Variables

Variables in Python are created by assignment and require no explicit type declaration. A variable is simply a name bound to an object in memory.

```
age = 25
price = 19.99
name = "Alice"
is_active = True
nothing = None
```

3.2 Built-in Data Types

Type	Example	Description
int	42, -7	Whole numbers of arbitrary size
float	3.14, -0.5	Decimal / floating-point numbers
str	"hello"	Immutable sequence of Unicode characters
bool	True, False	Boolean values
list	[1, 2, 3]	Ordered, mutable collection
tuple	(1, 2, 3)	Ordered, immutable collection
dict	{"a": 1}	Key-value mapping
set	{1, 2, 3}	Unordered collection of unique items
NoneType	None	Represents the absence of a value

3.3 Operators

Arithmetic Operators

```
a, b = 7, 2
a + b    # 9    addition
a - b    # 5    subtraction
a * b    # 14   multiplication
a / b    # 3.5  true division (always float)
a // b   # 3    floor division
a % b    # 1    modulo (remainder)
a ** b   # 49   exponentiation
```

Comparison and Logical Operators

```
a == b    # equal to
a != b    # not equal to
a > b     # greater than
```

```
a <= b    # less than or equal to
a and b   # logical AND
a or b    # logical OR
not a     # logical NOT
```

4. Strings and String Formatting

Strings are immutable sequences of characters. Python provides a rich set of methods for manipulating text.

```
s = "Hello, World!"
s.lower()          # 'hello, world!'
s.upper()          # 'HELLO, WORLD!'
s.replace("H", "J")# 'Jello, World!'
s.split(", ")      # ['Hello', 'World!']
s.strip()          # removes leading/trailing whitespace
len(s)            # 13
s[0]              # 'H' (indexing)
s[0:5]            # 'Hello' (slicing)
s[::-1]           # '!dlrow ,olleH' (reversed)
```

4.1 f-strings (Formatted String Literals)

Introduced in Python 3.6, f-strings are the preferred way to embed expressions inside string literals.

```
name = "Alice"
age = 30
print(f"{name} is {age} years old")
# Alice is 30 years old

pi = 3.14159
print(f"Pi rounded: {pi:.2f}")
# Pi rounded: 3.14
```

5. Collections: Lists, Tuples, Sets, and Dictionaries

5.1 Lists

Lists are ordered, mutable sequences that can hold mixed types.

```
fruits = ["apple", "banana", "cherry"]
fruits.append("date")      # add to end
fruits.insert(1, "kiwi")   # insert at index
fruits.remove("banana")    # remove by value
fruits.pop()               # remove and return last item
fruits.sort()              # sort in place
len(fruits)                # number of items

# List comprehension
squares = [x**2 for x in range(10)]
```

5.2 Tuples

Tuples are like lists but immutable, meaning their contents cannot be changed after creation. They are often used for fixed collections of values.

```
point = (3, 4)
x, y = point    # unpacking
point[0]        # 3
```

5.3 Sets

Sets store unique, unordered items and support mathematical set operations.

```
a = {1, 2, 3}
b = {2, 3, 4}
a | b    # union:      {1, 2, 3, 4}
a & b    # intersection: {2, 3}
a - b    # difference:  {1}
```

5.4 Dictionaries

Dictionaries map unique keys to values and are widely used for structured data.

```
person = {"name": "Alice", "age": 30}
person["email"] = "alice@example.com" # add a key
person.get("age")                      # 30
person.keys()                          # dict_keys([...])
person.values()                        # dict_values([...])
for key, value in person.items():
    print(key, value)
```

6. Control Flow: Conditionals and Loops

6.1 if / elif / else

```
temperature = 75

if temperature > 85:
    print("It's hot")
elif temperature > 60:
    print("It's mild")
else:
    print("It's cold")
```

6.2 for Loops

```
for i in range(5):
    print(i)          # 0 1 2 3 4

for fruit in ["apple", "banana"]:
    print(fruit)

for index, value in enumerate(["a", "b", "c"]):
    print(index, value)
```

6.3 while Loops

```
count = 0
while count < 3:
    print(count)
    count += 1
```

6.4 break, continue, and pass

- **break** exits the nearest enclosing loop immediately.
- **continue** skips the rest of the current iteration and moves to the next.
- **pass** does nothing; it is a placeholder where a statement is syntactically required.

7. Functions

Functions are defined with the `def` keyword and can accept parameters and return values.

```
def add(a, b):
    return a + b

def greet(name, greeting="Hello"):
    """Return a greeting message. `greeting` has a default value."""
    return f"{greeting}, {name}!"

greet("Alice")          # "Hello, Alice!"
greet("Bob", "Hi")      # "Hi, Bob!"
```

7.1 *args and **kwargs

Functions can accept a variable number of positional or keyword arguments.

```
def total(*numbers):
    return sum(numbers)

total(1, 2, 3)    # 6

def describe(**info):
    for key, value in info.items():
        print(f"{key}: {value}")

describe(name="Alice", age=30)
```

7.2 Lambda Functions

Lambdas are small, anonymous, single-expression functions.

```
square = lambda x: x ** 2
square(5)    # 25

names = ["Charlie", "Al", "Bob"]
names.sort(key=lambda n: len(n))
```

8. Modules and Packages

A module is simply a Python file containing definitions and code that can be imported into other files. A package is a directory of modules containing an `__init__.py` file.

```
# math_utils.py
def double(x):
    return x * 2

# main.py
import math_utils
math_utils.double(5)    # 10

from math_utils import double
double(5)                # 10

import math_utils as mu
mu.double(5)             # 10
```

8.1 Common Standard Library Imports

```
import math
import random
import datetime
import os
import sys
import json
```

9. Object-Oriented Programming

Python supports classes and objects, enabling encapsulation, inheritance, and polymorphism.

```
class Animal:
    def __init__(self, name, sound):
        self.name = name
        self.sound = sound

    def speak(self):
        return f"{self.name} says {self.sound}"

class Dog(Animal):
    def __init__(self, name):
        super().__init__(name, "Woof")

rex = Dog("Rex")
print(rex.speak())    # "Rex says Woof"
```

9.1 Key OOP Concepts

- **Encapsulation:** bundling data and methods that operate on that data within a class.
- **Inheritance:** a class (subclass) can inherit attributes and methods from another class (superclass), as shown with `Dog(Animal)` above.
- **Polymorphism:** different classes can implement the same method name in different ways, allowing objects to be used interchangeably.
- **Special (dunder) methods:** methods like `__init__`, `__str__`, and `__eq__` let custom classes integrate with built-in Python behavior.

10. Exception Handling

Exceptions are Python's mechanism for handling errors gracefully at runtime, using `try`, `except`, `else`, and `finally` blocks.

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Error: {e}")
else:
    print("No error occurred")
finally:
    print("This always runs")
```

10.1 Raising Exceptions

```
def withdraw(balance, amount):
    if amount > balance:
        raise ValueError("Insufficient funds")
    return balance - amount
```

10.2 Common Built-in Exceptions

Exception	Raised When...
ValueError	A function receives an argument of correct type but invalid value
TypeError	An operation is applied to an object of an inappropriate type
KeyError	A dictionary key is not found
IndexError	A sequence index is out of range
FileNotFoundError	A file or directory is requested but doesn't exist
ZeroDivisionError	Division or modulo operation with a divisor of zero

11. File Input and Output

Python's built-in `open()` function reads and writes files. Using a `with` block ensures the file is properly closed automatically.

```
# Writing to a file
with open("notes.txt", "w") as f:
    f.write("Hello, file!\n")

# Reading a file
with open("notes.txt", "r") as f:
    content = f.read()
    print(content)

# Reading line by line
with open("notes.txt", "r") as f:
    for line in f:
        print(line.strip())
```

11.1 File Modes

Mode	Meaning
"r"	Read (default); error if file doesn't exist
"w"	Write; creates file or overwrites existing content
"a"	Append; adds to the end of the file
"x"	Exclusive creation; fails if the file already exists
"b"	Binary mode (combine with above, e.g. "rb")

12. Useful Standard Library Modules

12.1 math

```
import math
math.sqrt(16)      # 4.0
math.pi            # 3.141592653589793
math.floor(4.7)    # 4
math.ceil(4.2)     # 5
```

12.2 random

```
import random
random.randint(1, 10)    # random int between 1 and 10
random.choice(["a", "b"]) # random element from a list
random.shuffle(my_list)  # shuffle a list in place
```

12.3 datetime

```
from datetime import datetime
now = datetime.now()
print(now.strftime("%Y-%m-%d %H:%M:%S"))
```

12.4 json

```
import json
data = {"name": "Alice", "age": 30}
json_str = json.dumps(data)    # dict -> JSON string
parsed = json.loads(json_str)  # JSON string -> dict
```

12.5 os and sys

```
import os
os.getcwd()          # current working directory
os.listdir(".")       # list files in a directory
os.path.join("a", "b") # 'a/b'

import sys
sys.argv              # list of command-line arguments
```

12.6 collections

```
from collections import Counter, defaultdict

Counter(["a", "b", "a", "c", "a"])
# Counter({'a': 3, 'b': 1, 'c': 1})

d = defaultdict(int)
d["x"] += 1  # no KeyError, defaults to 0 first
```

13. Virtual Environments and Package Management

Virtual environments create isolated Python installations per project, preventing dependency conflicts between projects.

```
# Create a virtual environment
$ python3 -m venv venv

# Activate it (macOS/Linux)
$ source venv/bin/activate

# Activate it (Windows)
> venv\Scripts\activate

# Install a package
(venv) $ pip install requests

# Save dependencies
(venv) $ pip freeze > requirements.txt

# Install from a requirements file
(venv) $ pip install -r requirements.txt
```

Note: *It is good practice to keep a `requirements.txt` or a `pyproject.toml` file in every project so dependencies can be reproduced on another machine.*

14. Best Practices and Style (PEP 8)

PEP 8 is Python's official style guide. Following consistent style makes code easier to read and maintain across teams.

- Use 4 spaces per indentation level (never tabs).
- Limit lines to a maximum of 79-99 characters depending on team convention.
- Use snake_case for functions and variables, and PascalCase for class names.
- Use UPPER_CASE for constants.
- Add two blank lines between top-level function and class definitions.
- Write docstrings for public modules, functions, classes, and methods.
- Prefer explicit imports over wildcard imports (`from module import *`).

```
# Good style
class BankAccount:
    """Represents a simple bank account."""

    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        """Add funds to the account."""
        self.balance += amount
```

15. Quick Reference Tables

15.1 Common Built-in Functions

Function	Purpose
len(obj)	Number of items in a collection
type(obj)	The type of an object
range(n)	Sequence of numbers from 0 to n-1
sorted(iterable)	Return a new sorted list
sum(iterable)	Sum of all items
min(iterable)	Smallest item
max(iterable)	Largest item
zip(a, b)	Pair up items from two iterables
map(func, iterable)	Apply a function to every item
filter(func, iterable)	Keep items where func returns True
isinstance(obj, type)	Check an object's type

15.2 String Formatting Cheat Sheet

```
f"{value}"           # basic insertion
f"{value:.2f}"       # 2 decimal places
f"{value:>10}"        # right-align in 10 spaces
f"{value:<10}"        # left-align in 10 spaces
f"{value:^10}"       # center-align in 10 spaces
f"{value:,}"         # thousands separator
f"{value:x}"         # hexadecimal
```

15.3 Comprehension Patterns

```
# List comprehension
[x*2 for x in range(5)]

# List comprehension with condition
[x for x in range(10) if x % 2 == 0]

# Dictionary comprehension
{x: x**2 for x in range(5)}

# Set comprehension
{x % 3 for x in range(10)}

# Generator expression (lazy evaluation)
```

```
(x*2 for x in range(5))
```

This guide is a condensed, original reference summarizing core Python concepts and is not a reproduction of the official Python documentation. For complete and authoritative details, consult the official docs at docs.python.org.